IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

A Mechanically Verified Translation of $\mathrm{MTL}_{0,\infty}$ into Timed Büchi Automata

ELIE FARES[1], JEAN-PAUL BODEVEIX[2], HUSSAIN AL-AQRABI[1] AND AZAR SALAMI[1]
[1]Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)
[2]IRIT, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

Corresponding author: Elie Fares (e-mail: efares@hct.ac.ae).

ABSTRACT Translating real-time specifications into timed automata supports automated analysis only if the translation preserves the timed-word language. We present a Rocq-checked soundness-and-completeness development for the pointwise dense-time translation of Metric Temporal Logic ($\mathrm{MTL}_{0,\infty}$) into timed Büchi automata (TBAs) via clocked Linear Temporal Logic. The encoding into clocked LTL is proved without assumptions on the backend; correctness of the resulting TBA holds relative to a backend language-equivalence contract. Earlier constructions extend the tableau LTL-to-Büchi translation with clocks directly and were proved on paper; here the translation instead factors through an untimed clocked-LTL formula that any standard LTL-to-Büchi backend can process. To the best of our knowledge, this factoring is new, and no published machine-checked development covers a pointwise $\mathrm{MTL}_{0,\infty}$-to-TBA construction in this semantic setting. Rocq checks all eight primitive cases, handling shared clocks and overlapping activations through soundness for every clock-consistent reset trace and completeness for one canonical reset trace. Relaxation and reset completion are also checked, giving a compositional theorem under an explicit language-equivalence contract for the Büchi backend, required on action-consistent words only. This contract is a premise (the convenience theorem postulates it as an axiom); Spot, graph import, parsing, printing, extraction, and zero-bound preprocessing lie outside the proof. The theorem assumes real-valued bounds, and strictly increasing divergent timestamps. On 31 benchmark configurations and 100 random formulas with integer bounds, TChecker inclusion and self-consistency checks find no counterexample on the 22 configurations that complete within the limits, and the backend contract is tested against ltl2ba without discrepancy on 28 configurations.

INDEX TERMS Metric temporal logic, timed Büchi automata, Rocq, formal verification, clocked LTL, semantic preservation.

I. INTRODUCTION

Reliable translation from real-time Metric Temporal Logic (MTL) to timed Büchi automata (TBAs) supports model checking and controller synthesis. Its correctness is the semantic link between a temporal specification and the automaton analyzed by a downstream tool. A bounded-response requirement, for example, is

$$\Box(req \Rightarrow \Diamond_{\leq d}\, ack).$$

Overlapping requests can leave several response obligations outstanding while sharing the clock assigned to one primitive timed formula. Table 1 illustrates why the reset semantics needs a correctness argument.

The mathematical construction studied here has substantial predecessors. Bulychev et al. establish exact timed-automaton language results for safety and co-safety fragments. Li et al. establish full-fragment $\mathrm{MTL}_{0,\infty}$-to-TBA language equality and show that their best-choice reset policy preserves language [1], [2]. Their work already contains the central timed-subformula clocking and reset architecture. They build the timed Büchi automaton directly, by extending the classical tableau translation with clocks. We keep that clocking and reset architecture but reformulate it: the formula is encoded into an untimed clocked-LTL formula, a standard LTL-to-Büchi backend is applied through an explicit contract, and relaxation and reset completion restore the timed meaning. Our contribution is this reformulation and its machine-checked proof. Table 6 compares these constructions.

Mathematical proofs and machine-checked proofs provide different assurance. A proof assistant requires the

syntax, timed-word semantics, clock updates, invariants, quantifiers, and proof dependencies to be represented in a kernel-checkable development. Here that formalization must also reconcile propositions of clocked Linear Temporal Logic (LTL) with genuine clock valuations. Related proof-assistant work addresses neighboring problems: Xu and Miao present a Metric Interval Temporal Logic (MITL)-to-fair-timed-automaton construction and discuss its implementation in the Prototype Verification System (PVS); Roohi and Viswanathan mechanize results for MITL decision procedures in PVS; Schimpf et al. verify an LTL-to-Büchi construction in Isabelle's higher-order logic (HOL); Wang et al. verify a Metric Linear Temporal Logic (MLTL)-to-regular-expression transformation in Isabelle/HOL; and Chattopadhyay and Mamouras mechanize online monitoring for past-time MTL under discrete-time quantitative semantics [3]–[7].

Several interacting proof obligations make the mechanization nontrivial. Soundness must hold for every clock-consistent reset trace; completeness must construct one trace that supports every dynamic activation of a shared, formula-keyed clock. Independent witnesses per activation cannot establish this single-trace property. The proof maintains residual obligations for upper- and lower-bound Until and Release, and uses occurrence-indexed structural induction so recursion through a syntax path retains the clock identity of an identical primitive formula. In addition, the propositional backend can assign clock atoms independently of actual clock valuations, so relaxation and reset completion must preserve the precise implication directions before the automaton theorem can be composed.

The principal contributions are:

- a reformulation of the construction that factors the translation through an untimed clocked-LTL formula and a standard LTL-to-Büchi backend, with the backend as an explicit language contract, in contrast with constructions that extend the tableau translation with clocks directly or that build networks of timed testers; to the best of our knowledge, this is the first presentation of the translation in this form, and the first machine-checked proof of a pointwise $\mathrm{MTL}_{0,\infty}$-to-TBA construction in this semantic setting;
- Rocq-checked soundness and completeness of the clocked-LTL encoding, including all eight hatted primitive cases and correctness at every syntactic occurrence;
- a canonical completeness construction that handles overlapping activations with one reset trace for each formula-keyed clock, together with policy-independent soundness for arbitrary consistent reset traces;
- checked relaxation and reset completion, composed with an explicit backend contract on action-consistent propositional words in MTL_to_TBA_correct_with;
- a concrete alarm-response instance whose manually transcribed Rocq backend satisfies the contract, with proved accepted-overlap and rejected-violation traces, and an inspectable proof and assumptions artifact; and
- an evaluation of the automaton produced by the extracted construction, before any optimization: sizes against CASAAL, language-inclusion checks with TChecker, agreement tests on random timed words, and tests of the backend contract (Section VI).

The result is relative to the formal model defined here: real-valued bounds, well-formed core formulas, strictly increasing divergent timestamps, event-guarded TBAs without location invariants. This differs from Li et al.'s integer-bound, nondecreasing-time, zero-initial semantics and is not a direct refinement of CASAAL's input/output semantics. The TBA theorem composes through an explicit backend-equivalence premise; for the alarm instance Rocq checks a manually transcribed backend record, not Spot or the graph-import path. Consequently, no automaton emitted by `mtl2tba` is covered by a machine-checked guarantee; the contract is only tested on the evaluated inputs (Section VI). The development proves a semantic theorem; the evaluated tool runs code extracted from its definitions, and extraction, parsing, and printing are outside the proof. Section V-F summarizes the theorem dependencies and assumptions. Optimization of the resulting automata and their export to UPPAAL are outside the scope of this article.

Section II defines the semantics and target model; Sections III–IV present the encoding and automaton construction; Section V develops the proof and includes the alarm-response instance in Section V-G; Section VI evaluates the construction, Section VII describes the artifact and reproduction procedure, and Section VIII discusses related work before the conclusion.

II. LOGIC AND SEMANTIC MODEL

A. POINTWISE $\mathrm{MTL}_{0,\infty}$

Let Σ be a finite set of actions. Formulas are in negation normal form and use Boolean operators, next, untimed Until and Release, and timed Until and Release whose intervals have one endpoint at zero or infinity. We write a bound as $\sim d$, with $\sim \in \{<, \leq, >, \geq\}$. The logic also contains the hatted operators $\widehat{\mathsf{U}}_{\sim d}$ and $\widehat{\mathsf{R}}_{\sim d}$. In a hatted operator the current position is skipped by the logical interval, while its timestamp remains the timing reference. We use $\widehat{\Box}_{\sim d}\varphi$ as shorthand for $\bot\, \widehat{\mathsf{R}}_{\sim d}\, \varphi$; it requires φ at each strictly future position whose elapsed time from the current position satisfies the bound. Dually, $\widehat{\Diamond}_{\sim d}\varphi$ abbreviates $\top\, \widehat{\mathsf{U}}_{\sim d}\, \varphi$.

A timed word is $w = (a_i, t_i)_{i\in\mathbb{N}}$, where $a_i \in \Sigma$, $0 \leq t_0 < t_1 < \cdots$, and timestamps diverge. We write $\mathsf{elap}_w(i,j) = t_j - t_i$. For Until, $w, i \models_{\mathrm{MTL}} p\, \mathsf{U}_{\sim d}\, q$ holds when some $j \geq i$ satisfies q and $t_j - t_i \sim d$, with p at each position in $[i,j)$. For hatted Until, the witness is $j > i$ and p is required on (i,j). Release is the dual universal condition: at every position in the bounded region, either q holds or p held earlier in the corresponding interval. Hatted Release excludes the current position from that region. We use the standard pointwise semantics [8].

For precision, the four bounded clauses used below are:

$$
\begin{aligned}
w,i \models_{\mathrm{MTL}} p\ \mathsf{U}_{\sim d}\ q &\iff \exists j \geq i.\ w,j \models_{\mathrm{MTL}} q \\
&\quad \wedge\ t_j - t_i \sim d \\
&\quad \wedge\ \forall k \in [i,j).\ w,k \models_{\mathrm{MTL}} p, \\
w,i \models_{\mathrm{MTL}} p\ \widehat{\mathsf{U}}_{\sim d}\ q &\iff \exists j > i.\ w,j \models_{\mathrm{MTL}} q \\
&\quad \wedge\ t_j - t_i \sim d \\
&\quad \wedge\ \forall k \in (i,j).\ w,k \models_{\mathrm{MTL}} p, \\
w,i \models_{\mathrm{MTL}} p\ \mathsf{R}_{\sim d}\ q &\iff \forall j \geq i. \\
&\quad \Big(t_j - t_i \sim d \Rightarrow \\
&\quad\quad \Big(w,j \models_{\mathrm{MTL}} q \\
&\quad\quad\quad \vee\ \exists k \in [i,j).\ w,k \models_{\mathrm{MTL}} p\Big)\Big), \\
w,i \models_{\mathrm{MTL}} p\ \widehat{\mathsf{R}}_{\sim d}\ q &\iff \forall j > i. \\
&\quad \Big(t_j - t_i \sim d \Rightarrow \\
&\quad\quad \Big(w,j \models_{\mathrm{MTL}} q \\
&\quad\quad\quad \vee\ \exists k \in (i,j).\ w,k \models_{\mathrm{MTL}} p\Big)\Big).
\end{aligned}
$$

The index intervals select event positions, whereas $t_j - t_i$ measures elapsed physical time. Thus, when a bound falls between two events, no event occurs exactly at that bound, and interval membership is still evaluated at the event positions. The strict increase assumption excludes simultaneous events; divergence is used in the lower-bounded Until fairness argument.

The Rocq development represents action names by natural numbers rather than a parameterized finite type. Each formula mentions finitely many names, so its semantics and generated labels distinguish actions only by equality or inequality with those names.

The well-formedness condition requires $d > 0$ for every lower bound and strict upper bound, and $d \geq 0$ for a non-strict upper bound. Ordinary timed operators are definitions in the mechanized core. For upper bounds $\precsim \in \{\leq, <\}$ and lower bounds $\succsim \in \{\geq, >\}$, they are:

$$p\ \mathsf{U}_{\precsim d}\ q \equiv q \vee (p \wedge (p\ \widehat{\mathsf{U}}_{\precsim d}\ q)), \tag{1}$$

$$p\ \mathsf{U}_{\succsim d}\ q \equiv p \wedge (p\ \widehat{\mathsf{U}}_{\succsim d}\ q), \tag{2}$$

$$p\ \mathsf{R}_{\precsim d}\ q \equiv q \wedge (p \vee (p\ \widehat{\mathsf{R}}_{\precsim d}\ q)), \tag{3}$$

$$p\ \mathsf{R}_{\succsim d}\ q \equiv p \vee (p\ \widehat{\mathsf{R}}_{\succsim d}\ q). \tag{4}$$

These definitions separate the current-position case from the strictly future obligation. Their pointwise semantic compatibility, including the stated bound hypotheses, is proved in Rocq:

Theorem II.1 (Semantic compatibility of the derived timed operators)**.** *Under the well-formedness conditions above, Eqs.* (1)– (4) *agree with the standard pointwise semantics for all upper and lower bounds.*

The identities follow by separating the current-position case from a strictly future endpoint. For upper Until, a witness at i contributes q; every later witness requires p at i and leaves the hatted obligation. A lower bound with $d > 0$ excludes i, so p is required now and the remaining witness is hatted. The Release identities are the dual case split: an upper-bound region includes i and therefore requires q there, while a lower-bound region excludes i. Strict upper bounds also require $d > 0$, avoiding the degenerate < 0 case. The eight corresponding Rocq lemmas are mapped to their source modules and assumptions in Table 3. Thus only the eight hatted timed constructors require primitive translation clauses. If a source formula has a zero lower bound, strict increase of timestamps makes every strictly future elapsed time positive. At the hatted level, $\widehat{\mathsf{U}}_{\geq 0}$ and $\widehat{\mathsf{U}}_{>0}$ therefore reduce to $\mathsf{X}(p\ \mathsf{U}\ q)$, while $\widehat{\mathsf{R}}_{\geq 0}$ and $\widehat{\mathsf{R}}_{>0}$ reduce to $\mathsf{X}(p\ \mathsf{R}\ q)$. For ordinary operators, the current-position case must also be simplified: the ≥ 0 cases are the corresponding untimed Until and Release operators, and the > 0 cases reduce to $p \wedge \mathsf{X}(p \mathsf{U} q)$ and $p \vee \mathsf{X}(p \mathsf{R} q)$, respectively. These reductions rely on strictly increasing event times and must be applied before encoding a source formula as a Rocq core formula. A zero strict upper bound has an empty interval: Until reduces to $\bot$ and Release to $\top$; strict upper-bound constructors in the core consequently require $d > 0$. For a non-strict upper bound of zero, the hatted future operators reduce to $\widehat{\mathsf{U}}_{\leq 0} \equiv \bot$ and $\widehat{\mathsf{R}}_{\leq 0} \equiv \top$, because all strictly future events have positive elapsed time. The derived ordinary operators therefore both reduce to their right operand: $p \mathsf{U}_{\leq 0} q \equiv q$ and $p \mathsf{R}_{\leq 0} q \equiv q$. These reductions make the boundary explicit: the mechanized domain excludes zero lower bounds and zero strict upper bounds, but includes a non-strict upper bound of zero.

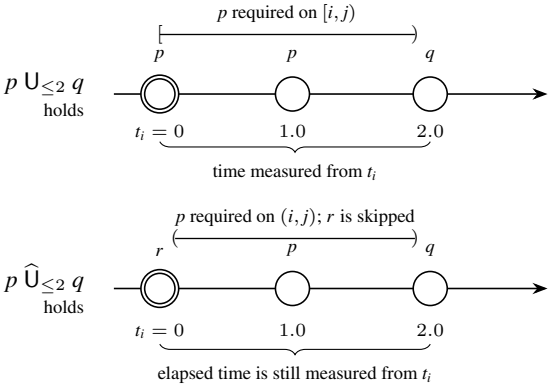


FIGURE 1. Ordinary and hatted Until. The double circle is the timing reference. The hatted operator skips the current event logically but measures from its timestamp.

Figure 1 contrasts the two readings. The distinction is useful for sporadicity: if consecutive occurrences of e must be separated by at least θ, one may write $\Box(e \Rightarrow \widehat{\Box}_{<\theta} \neg e)$. The hatted form retains the triggering event as the time origin; ordinary bounded temporal operators do not express this particular reference shift directly. Related pointwise MTL constructions and the CASAAL translation are discussed in prior work [2], [9]–[11].

B. TIMED BÜCHI AUTOMATA

The verified target is an event-guarded TBA $\mathcal{A} = (L, \ell_0, X, E, Acc)$. Here L is a finite set of locations, ℓ_0 the initial location, X a finite set of clocks, $E \subseteq L \times \mathsf{Lab} \times \mathsf{Guard} \times 2^X \times L$ is the set of action-labeled transitions with guards and reset sets, and $Acc \subseteq L$ the Büchi set. A run is a sequence $(\ell_i, v_i)_{i\geq 0}$ starting at ℓ_0; it follows one transition per event and visits Acc infinitely often. If the selected edge at event i is $(\ell_i, \alpha_i, g_i, Z_i, \ell_{i+1})$, then the event action must satisfy α_i and the valuation v_i must satisfy every guard in g_i. There are no location invariants in this model: timing constraints are checked at the events of the timed word. This is the TBA record represented in Rocq.

Let v_i be the clock valuation tested at event i and $\Delta_i = t_{i+1} - t_i$. A transition at event i tests v_i before its reset decision; the next valuation is

$$v_{i+1}(x) = \begin{cases} \Delta_i, & x \in Z_i, \\ v_i(x) + \Delta_i, & \text{otherwise.} \end{cases}$$

The initial nonnegative valuation v_0 is existentially quantified, as in the Rocq clock-consistent-extension semantics; standard zero initialization is not assumed.

III. CLOCKED-LTL TRANSLATION

A. SHARED CLOCK IDENTITIES

The translation is structural. It maps Boolean and untimed temporal operators to their LTL counterparts and maps each primitive timed subformula to an LTL formula over an extended alphabet. For each action a, the extended alphabet contains distinct action atoms $\mathsf{LAct}(a)$ and $\mathsf{LNAct}(a)$, as well as clock-related atoms: clock comparisons $x \sim d$, a reset atom $\mathsf{rst}(x)$, and a preservation atom $\mathsf{unch}(x)$. Under timed-word interpretation, the action atoms mean that the current action is a or is different from a, respectively. The reset and preservation atoms describe the clock update at the current event. Clock values are read before the event's reset, matching the guard semantics of a timed automaton.

For a formula f, let $Clock(f)$ be its finite set of distinct primitive hatted timed subformulas. The clock associated with $F \in Clock(f)$ is $x = F$ itself. Clock identity uses structural equality on the Rocq MTL syntax tree. The main result proves that this clock also serves all dynamic activations of its subformula. The clock count is therefore bounded by the number of distinct primitive timed subformulas, which is at most the number of timed occurrences in f. If structural subformulas are represented as shared nodes, the translation can share their LTL encodings as well; this does not remove the usual worst-case growth of an LTL-to-Büchi construction.

This clock-sharing policy is distinct from automaton-state minimization. The reset trace is chosen for each formula identity as a whole. In particular, the proof cannot justify the clock bound by assigning a fresh clock to each activation and subsequently identifying those clocks: it establishes directly that a single trace of reset decisions for the formula is sufficient.

TABLE 1. A finite prefix illustrating overlapping response obligations. The deadline is five time units; the acknowledgment at time six is late for the request at time zero even though it is age two from the later request.

Event i	Action	Time t_i	Oldest age	Reset age
0	*req*	0	0	0
1	*req*	4	4	4
2	*ack*	6	6	2

B. EIGHT PRIMITIVE CLAUSES

Let F be a primitive hatted formula at path π, with operands p, q. Write $x = \mathsf{clock_of}(F)$, $P = \mathcal{T}_{\pi 0}(p)$, $Q = \mathcal{T}_{\pi 1}(q)$, and define $\alpha \mathsf{W} \beta = (\alpha \mathsf{U} \beta) \vee \mathsf{G}\alpha$. For upper comparisons $\precsim \in \{\leq, <\}$ and lower comparisons $\succsim \in \{\geq, >\}$, complements are $\leq^{\mathrm{c}} = >$, $<^{\mathrm{c}} = \geq$, $\geq^{\mathrm{c}} = <$, and $>^{\mathrm{c}} = \leq$. The four clause shapes give all eight primitive cases:

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{U}}_{\precsim d}\, q) = \mathsf{X}\big(((x \precsim d) \wedge \mathsf{unch}(x) \wedge P) \mathsf{U}\, ((x \precsim d) \wedge Q)\big), \tag{5}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{U}}_{\succsim d}\, q) = \mathsf{rst}(x) \wedge \mathsf{X}\big((P \mathsf{U}\, ((x \succsim d) \wedge (P \mathsf{U}\, Q))) \vee (\mathsf{G}P \wedge \mathsf{GF}Q)\big), \tag{6}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{R}}_{\precsim d}\, q) = \mathsf{rst}(x) \wedge \mathsf{X}\big(Q \mathsf{W}\, ((x \precsim^{\mathrm{c}} d) \vee (P \wedge Q))\big), \tag{7}$$

$$\mathcal{T}_\pi(p\, \widehat{\mathsf{R}}_{\succsim d}\, q) = \mathsf{X}\big(((x \succsim^{\mathrm{c}} d) \wedge \mathsf{unch}(x)) \mathsf{W}\, ((P \mathsf{R}\, Q) \vee ((x \succsim^{\mathrm{c}} d) \wedge P))\big). \tag{8}$$

The outer X implements the hatted shift; $\mathsf{rst}(x)$ and $\mathsf{unch}(x)$ encode the restart and preservation choices. Until uses strong Until for a bounded witness and weak Until for the Release window. Lower-bounded Until also has a fairness branch for an obligation whose threshold is delayed indefinitely. Every clock atom occurs positively, which is the property used by relaxation in Section IV-B. The eight local proof arguments establish the precise behavior in Section V-A.

C. SHARED CLOCKS UNDER OVERLAPPING ACTIVATIONS

Hatted operators preserve the triggering timestamp while moving logical evaluation to the next event. For example, $\Box(e \Rightarrow \widehat{\Box}_{<\theta} \neg e)$ measures an exclusion window from the triggering event. The leading next in an ordinary operator would shift that reference, while omitting it also constrains the triggering event. Fig. 2 contrasts these readings.

The proof assigns one clock to each structurally distinct primitive timed formula, even when that formula is activated repeatedly. The finite prefix in Table 1 illustrates why completeness must account for all pending obligations: resetting a single clock at the later request can make an expired earlier deadline appear to be met. The semantic proof below constructs one trace per formula-keyed clock and establishes the residual invariants needed when several activations are pending at once.

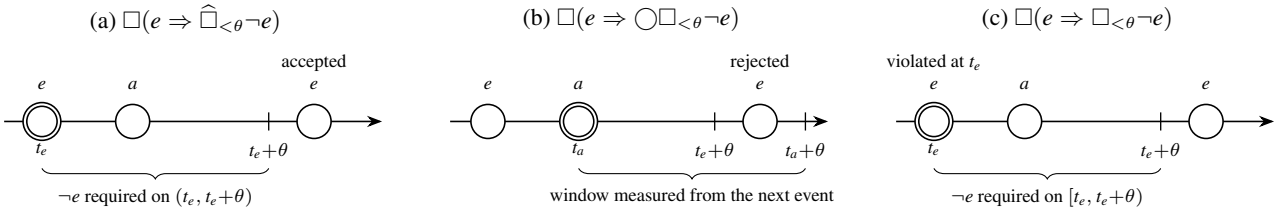


FIGURE 2. The hatted sporadicity property and two ordinary-operator substitutions. A leading next shifts the timing reference to the following event; omitting it includes the triggering event in the window.

IV. FROM CLOCKED LTL TO A TBA

A. LTL-TO-BÜCHI CONTRACT

The backend treats the clock-related atoms as independent propositions and receives the negated action atom $\mathsf{LNAct}(a)$ as the negation of the action atom $\mathsf{LAct}(a)$. It returns a propositional Büchi automaton (PBA) whose language is exactly that of the clocked-LTL formula. The theorem is stated for any such backend, deterministic or not. A propositional word is *action-consistent* when, at every position, $\mathsf{LNAct}(a)$ holds exactly when $\mathsf{LAct}(a)$ does not.

Assumption IV.1 (Backend correctness)**.** *For each formula F and each action-consistent propositional word s over its atoms,* $\mathsf{PBA_accepts}(A, s)$ *iff* $s, 0 \models F$*, where A is the backend automaton for F.*

This contract ranges over all action-consistent propositional words, including assignments of the clock-related atoms that do not correspond to any clock-consistent timed word. That scope is used by relaxation.

B. RELAXATION AND RESET COMPLETION

The propositional backend regards every clock-related atom as independent. Its correctness contract therefore applies to all action-consistent Boolean words, including assignments of the clock-related atoms that cannot arise from any clock-consistent timed trace. Every clock-related atom in the translation occurs positively. Relaxation acts on each backend transition cube (conjunction of atomic constraints) by removing negative literals over clock comparisons and reset/preservation markers, removing irrelevant clock-related literals, and discarding contradictory cubes. Action literals and positive relevant clock literals are retained. This weakens the cubes without asking the backend to understand clocks.

Reset completion converts each relaxed cube to a timed transition. Clocks of upper-bounded hatted Until and lower-bounded hatted Release are preservation clocks (their clauses use unch); clocks of lower-bounded hatted Until and upper-bounded hatted Release are restart clocks (their clauses use rst). A preservation clock is reset unless the cube contains $\mathsf{unch}(x)$; a restart clock is reset exactly when the cube contains $\mathsf{rst}(x)$. The markers are then removed, clock comparisons become guards, and action literals become labels. The two clock classes move in opposite directions, so the proof treats them separately.

Lemma IV.2 (Reset completion)**.** *Let an accepting run of the relaxed backend be accompanied by a clock-consistent extension ρ of its timed word. Reset completion defines a clock-consistent extension ρ' with the same timed word and the same initial clock valuation. The same state sequence is an accepting run of the completed TBA on ρ'.*

Proof. Fix a selected backend cube at position i. For a preservation clock x, if the cube contains $\mathsf{unch}(x)$ then both traces preserve x. Otherwise the completed TBA resets x, while ρ may either reset it or preserve it. Thus the completed reset set contains the original reset decision for x. Induction on positions using the clock recurrence gives $\mathsf{val}_{\rho'}(i,x) \le \mathsf{val}_{\rho}(i,x)$. These clocks occur only in upper tests ($x \precsim d$), which remain true when the value decreases.

For a restart clock x, completion resets it exactly when the cube contains $\mathsf{rst}(x)$. A true reset marker requires that ρ resets x; without the marker completion preserves it, whether or not ρ resets it. The completed reset set is therefore contained in the original one, and the same induction gives $\mathsf{val}_{\rho'}(i,x) \ge \mathsf{val}_{\rho}(i,x)$. These clocks occur only in lower tests ($x \succsim d$, or their equivalent window-ending comparisons), which remain true when the value increases. The clock recurrence holds by the construction of ρ'. State transitions, event labels, guards that remain true, and the Büchi accepting set are unchanged, so the run remains accepting. Rocq proves this result as complete_complete. □

C. WORKED EXAMPLE: BOUNDED EVENTUALITY WITH AN EXCLUSION WINDOW

Let

$$f = (\mathsf{F}_{\le 5} p) \wedge (\mathsf{G}_{<2} \neg p).$$

It requires p to be false at the origin and until age two, then true by age five. Fig. 3 shows a hand-simplified equivalent one-clock TBA, not the syntax-directed output, which assigns separate clocks to the two distinct timed subformulas. The steps below show the clocked-LTL translation and timed reinterpretation. The concrete instance checked in Rocq is the alarm-response example in Section V-G.

Step 1: The formula is in negation normal form. Expanding the derived operators and applying the timed-

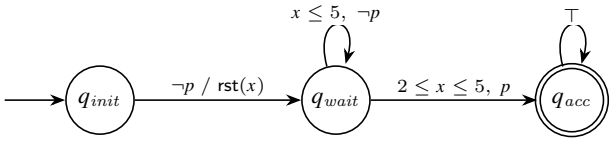


FIGURE 3. Hand-simplified equivalent event-guarded TBA for f. The initial transition requires $\neg p$ and resets x; in the waiting location, $x \le 5$, and acceptance also requires $x \ge 2$. Thus p first holds at age in $[2, 5]$.

operator identities gives

$$\begin{aligned} f &= (\top \,\mathsf{U}_{\le 5}\, p) \wedge (\bot \,\mathsf{R}_{<2}\, \neg p) \\ &= \big(p \vee (\top \wedge \top\, \widehat{\mathsf{U}}_{\le 5}\, p)\big) \\ &\qquad \wedge \big(\neg p \wedge (\bot \vee \bot\, \widehat{\mathsf{R}}_{<2}\, \neg p)\big). \end{aligned}$$

These current-position cases follow Eqs. (1) and (3).

Step 2: The two distinct primitive timed subformulas receive clocks x and y, respectively. Their clocked-LTL translation is

$$\begin{aligned} \Phi_U &= p \vee \Big(\top \wedge \mathsf{X}\Big(((x \le 5) \wedge \mathsf{unch}(x) \wedge \top)\mathsf{U} \\ &\qquad ((x \le 5) \wedge p)\Big)\Big), \\ \Phi_R &= \neg p \wedge \big(\bot \vee (\mathsf{rst}(y) \wedge \\ &\qquad \mathsf{X}(\neg p\, \mathsf{W}\, ((y \ge 2) \vee (\bot \wedge \neg p)))))\big), \\ \mathcal{T}(f) &= \Phi_U \wedge \Phi_R. \end{aligned}$$

The Until clause preserves x; the Release test $y \ge 2$ complements the source bound < 2, allowing p from age two.

Step 3: The Spot 2.16 graph for $\mathcal{T}(f)$ is redrawn at the top of Fig. 4. The clock guards $x \le 5$ and $y \ge 2$, the markers $\mathsf{unch}(x)$ and $\mathsf{rst}(y)$, and the action atom p are propositions to the backend. The graph has four states and seven transitions, the size reported for F11 in Table 5.

Step 4: Relaxation removes negative literals over extended clock atoms, here $\neg(y \ge 2)$. Reset completion resets the preservation clock x unless $\mathsf{unch}(x)$ is present; the restart clock y is reset exactly when $\mathsf{rst}(y)$ is present. The bottom of Fig. 4 shows the same state graph after these operations: each edge carries the event, the clock guard, and the resets. The event is an action (p or $\neg p$), and $\top$ stands for any event. Relaxation removes $\neg(y \ge 2)$ from the loop on 2, which therefore stays enabled after age 2. From then on state 2 behaves as state 3, since y is never reset again, so no word is added; the two states could be merged, which the translation does not do.

V. MECHANIZED CORRECTNESS

For a formula f, an extended word ρ augments the timed word with a nonnegative value $\mathsf{val}_\rho(i,x)$ and a reset decision $\mathsf{reset}_\rho(i,x)$ for every formula-keyed clock $x \in \mathit{Clock}(f)$ at each position i. It is clock-consistent when, with $\Delta_i = t_{i+1} - t_i$,

$$\mathsf{val}_\rho(i+1,x) = \begin{cases} \Delta_i, & \mathsf{reset}_\rho(i,x), \\ \mathsf{val}_\rho(i,x) + \Delta_i, & \text{otherwise.} \end{cases} \tag{9}$$

The value at i is tested before the reset decision at i. The relation $\mathsf{same_base}(\rho, w)$ means that the projection of ρ onto actions and timestamps gives w.

A. SOUNDNESS FOR ARBITRARY RESET TRACES

The soundness direction does not select or assume any particular reset policy; in particular, it does not use the canonical policy that Section V-B defines for completeness. For each primitive formula F at path π, the corresponding Rocq lemma `sound_*` turns $\rho, i \models_{\mathrm{LTL}} \mathcal{T}_\pi(F)$ into $\mathsf{ew_base}(\rho), i \models_{\mathrm{MTL}} F$, assuming clock consistency and induction hypotheses for its operands. Clock consistency gives two useful bounds. If a clock is preserved on $[i,j)$, then

$$\mathsf{val}_\rho(j,x) = \mathsf{val}_\rho(i,x) + \mathsf{elap}_w(i,j).$$

Since clock values are nonnegative, preservation implies $\mathsf{val}_\rho(j,x) \ge \mathsf{elap}_w(i,j)$. If it is reset at i, then for every $j \ge i+1$,

$$\mathsf{val}_\rho(j,x) \le \mathsf{elap}_w(i,j),$$

with equality when it is not reset again before j. The upper-guard implication follows from the first inequality; the lower-guard implication follows from the second. Both follow by induction from Eq. (9) and nonnegative delays.

Upper-bounded hatted Until. The outer X moves to the first strictly future event. The strong Until then supplies a witness $j > i$ satisfying Q, with P at every intermediate event and $\mathsf{unch}(x)$ throughout that prefix. Regardless of whether the clock was reset at i, its value at $i+1$ is at least Δ_i. Preservation after $i+1$ therefore gives

$$\mathsf{val}_\rho(j,x) \ge \Delta_i + \mathsf{elap}_w(i+1,j) = \mathsf{elap}_w(i,j).$$

The endpoint guard $x \le d$ implies $\mathsf{elap}_w(i,j) \le d$; the guard $x < d$ implies $\mathsf{elap}_w(i,j) < d$. Induction on P, Q yields the hatted Until witness and prefix in both cases. The strong Until is essential: an infinite continuation with no Q endpoint is not a bounded Until witness.

Lower-bounded hatted Until. Here the translation requires $\mathsf{rst}(x)$ at i. The finite branch (the first disjunct of Eq. (6)) waits for a position with $x \gtrsim d$ and then supplies an ordinary $P\,\mathsf{U}\,Q$ witness. Since subsequent resets can only make a clock younger, $\mathsf{val}_\rho(j,x) \le \mathsf{elap}_w(i,j)$; thus the threshold guard proves the required lower-bound comparison. The nested Until supplies P up to the Q witness. The other branch is $\mathsf{G}P \wedge \mathsf{GF}Q$. It gives an unbroken P prefix and unboundedly many Q positions; divergence of timestamps lets us choose a Q witness beyond any finite lower bound. For the strict case, choose a recurring Q at distance at least d from $i+1$. Since $t_{i+1} > t_i$, that same event is strictly more than d after i. This is the additional argument in sound_UhatGt; it preserves the strict boundary without changing the constant.

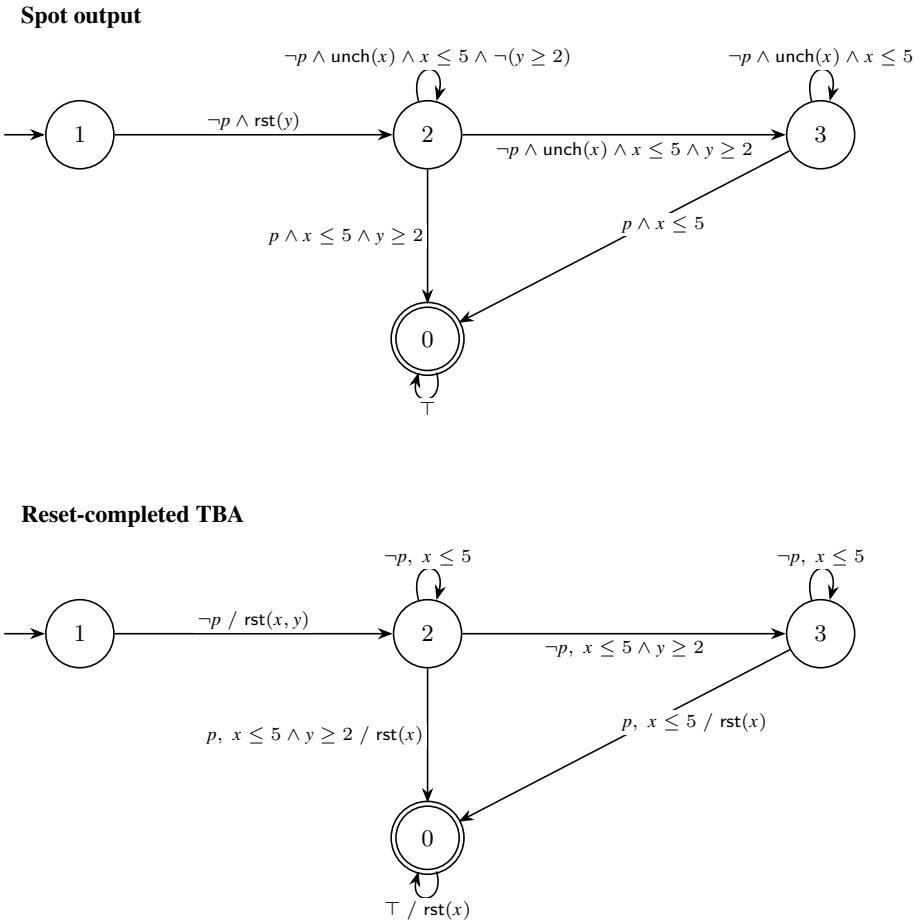


FIGURE 4. **The Spot 2.16 graph for $\mathcal{T}(f)$ (top, redrawn; the labels are propositional cubes over the action atom p, the clock guards, and the markers $\mathsf{unch}(x)$ and $\mathsf{rst}(y)$) and the reset-completed TBA obtained from it by hand (bottom; same state numbers). A TBA edge is labeled by the event, the clock guard, and, after a slash, the clocks reset; $\top$ is any event, and an edge without a slash resets no clock. This graph-to-TBA illustration is not the formally verified alarm-response backend instance.**

Upper-bounded hatted Release. The translation resets the clock at i and uses weak Until with Q as its left condition. Consequently, Q holds at each future event until either $P \wedge Q$ discharges the Release obligation or the complementary clock test marks the end of the upper window. For a non-strict upper bound, the window ends at $x > d$; for a strict upper bound, it ends at $x \geq d$. If the clock has been reset again, its value is no larger than the elapsed time from i, so either endpoint guard can hold only after the corresponding physical-time window has ended. If weak Until ends at $P \wedge Q$, P at that earlier event discharges later Release obligations. This proves both comparisons with their exact strict boundary.

Lower-bounded hatted Release. Before the threshold, the left side of weak Until requires both the pre-threshold comparison and $\mathsf{unch}(x)$. Thus an arbitrary reset trace cannot move the clock backwards while the pre-threshold condition remains active. Once the comparison becomes false, the translation requires ordinary $P \mathbin{\mathsf{R}} Q$ from that event, unless the right-side alternative has already terminated the obligation at a position satisfying P while still below the threshold. An older initial clock value can make the translation enter the ordinary Release branch earlier than the MTL interval does; this only imposes a stronger condition.

The standard LTL semantics used here are formalized in Rocq as LG_semantics, LGF_semantics, LW_semantics, and LF_semantics. Time divergence is used only for the fairness witness in lower-bounded Until; no assumption on the reset policy beyond clock consistency is used in any soundness family.

B. CANONICAL RESET POLICY AND RESIDUAL OBLIGATIONS

Completeness must choose one clock trace for each primitive formula, even when activations overlap. The canonical policy preserves a reference only while its residual obligation is still useful. Recall that $\mathsf{elap}_w(i,j) = t_j - t_i$. For upper-bounded Until define

$$\begin{aligned}\mathsf{URes}_{\precsim}(i,v,d,p,q) \iff\ & \exists j \geq i.\ v + \mathsf{elap}_w(i,j) \precsim d \\ & \wedge\ w,j \models_{\mathrm{MTL}} q \\ & \wedge\ \forall k \in [i,j).\ w,k \models_{\mathrm{MTL}} p \quad (10)\end{aligned}$$

TABLE 2. Canonical reset decision at position i, with $\precsim$ ranging over $\{\leq, <\}$ and $\succsim$ over $\{\geq, >\}$. A preservation row gives the condition under which the clock is not reset; otherwise it is reset.

Primitive formula F	Decision at i
$p\,\widehat{\mathsf{U}}_{\precsim d}\,q$	Preserve iff $v \precsim d$, $\mathsf{URes}_{\precsim}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} q$.
$p\,\widehat{\mathsf{U}}_{\succsim d}\,q$, $p\,\widehat{\mathsf{R}}_{\precsim d}\,q$	Reset iff $w,i \models_{\mathrm{MTL}} F$.
$p\,\widehat{\mathsf{R}}_{\succsim d}\,q$	Preserve iff $v \not\succsim d$, $\mathsf{RRes}_{\succsim}(i,v,d,p,q)$, and $w,i \not\models_{\mathrm{MTL}} p$.

The residual says that a q endpoint is still reachable with the current clock age and the required p prefix. For lower-bounded Release define

$$\mathsf{RRes}_{\succsim}(i,v,d,p,q) \iff \forall j \geq i.\ v + \mathsf{elap}_w(i,j) \not\succsim d \Rightarrow \Big(w,j \models_{\mathrm{MTL}} q \vee \exists k \in [i,j).\, w,k \models_{\mathrm{MTL}} p\Big) \quad (11)$$

The residual RRes is universal: every position already beyond the threshold must have q or an earlier p that discharges the obligation; URes is its existential counterpart (a reachable q endpoint). Instantiating $\precsim$ as $\leq$ or $<$, and $\succsim$ as $\geq$ or $>$, gives the non-strict and strict variants.

For a clock $x = F$, let $v = \mathsf{val}_{\rho^\star}(i,x)$; Table 2 gives the policy. The resulting values begin at $u_0 = 0$ and evolve by

$$u_{i+1} = \begin{cases} \Delta_i, & \text{if the policy resets at } i, \\ u_i + \Delta_i, & \text{otherwise.} \end{cases}$$

This is a word-dependent completeness witness, not a reset algorithm in an executable translator. Its decisions are keyed by the primitive formula itself, not by an occurrence path.

C. COMPLETENESS FOR ONE CANONICAL TRACE

For every formula satisfied by w, completeness constructs one clock-consistent extension $\rho^\star$ that serves all activations of every shared clock. The four operator families use different invariants:

Upper-bounded hatted Until. Choose the first position $j > i$ at which q holds with p on $[i,j)$, while the activation remains pending. The key invariant along the translated strong-Until prefix is

$$\mathsf{val}_{\rho^\star}(n,x) + \mathsf{elap}_w(n,j) \leq d \quad (i < n \leq j). \quad (12)$$

For $< d$, replace $\leq d$ by $< d$. The invariant need not hold at i: the translation begins with X, so the first clock test is at $i+1$. If the policy resets at i, then $u_{i+1} = \Delta_i$ and $u_{i+1} + \mathsf{elap}_w(i+1,j) = \mathsf{elap}_w(i,j)$, which is within the bound for the chosen witness. If it preserves at i, the residual supplies a still reachable q endpoint and p prefix; because the current q has not discharged that residual, its witness is strictly after i, and the same identity advances the residual inequality to $i+1$. Because j is the first such position, q fails at every intermediate position n with $i < n < j$: otherwise n would be an earlier witness, since p holds on $[i,n) \subseteq [i,j)$. The invariant gives $\mathsf{val}_{\rho^\star}(n,x) + \mathsf{elap}_w(n,j) \leq d$, so $\mathsf{URes}_{\leq}$ holds at n with witness j. Hence the policy preserves the reference at every intermediate position, which is what the conjunct $\mathsf{unch}(x)$ of the translation requires; the endpoint j satisfies q and the clock comparison. A reference reset earlier has a larger age than any later one, so if it still reaches the shared endpoint within d, the obligations of the later activations are covered as well. This is the invariant proved by complete_UhatLe and complete_UhatLt.

Lower-bounded hatted Until. Whenever the formula is true at an activation, the policy resets its clock. Starting from $i+1$, a first position at which the canonical clock reaches d (or strictly exceeds d) and has a qualifying later q witness gives the finite branch (the first disjunct of Eq. (6)): the clock guard supplies the lower-bound test, and the nested $P\,\mathsf{U}\,Q$ supplies the remaining witness. Before the first such position, the proof maintains an active-witness invariant. At each $n \geq i+1$ it supplies an activation $s < n$, a witness $m \geq n$ with $t_m - t_s \geq d$ and $w,m \models q,p$ at every intermediate position, and $\mathsf{val}_{\rho^\star}(n,x) = t_n - t_s$. Thus p holds at n and a q witness remains at or after every n. If a first qualifying position occurs, the finite branch applies; if none occurs, the invariant yields $\mathsf{G}P \wedge \mathsf{GF}Q$, the fairness branch (the second disjunct of Eq. (6)). Divergence converts recurring q positions into a witness beyond the lower bound. These are the arguments formalized by complete_UhatGe and complete_UhatGt.

Upper-bounded hatted Release. At every activation where the hatted Release formula is true, the policy resets the restart clock. The first future event must satisfy q, as required by the hatted Release semantics. The universal condition then continues as weak Until: q holds until either $p \wedge q$ discharges all later points or the clock passes the end of the bounded window. For a $\leq d$ window the stopping guard is $x > d$; for a $< d$ window it is $x \geq d$. Resetting at a later true activation can extend the clock window, but the later formula's truth supplies the corresponding future q-or-discharge condition. Thus a single reset trace satisfies both the old and new obligations. This proves complete_RhatLe and complete_RhatLt.

Lower-bounded hatted Release. While $\mathsf{RRes}_{\geq}$ holds, $v < d$, and p has not discharged the obligation, the policy preserves the clock. Preservation advances both the clock and the residual by

$$(i,v) \longmapsto (i+1, v+\Delta_i), \qquad \mathsf{elap}_w(i,j) = \Delta_i + \mathsf{elap}_w(i+1,j).$$

Substitution in Eq. (11) proves $\mathsf{RRes}_{\geq}(i+1, v+\Delta_i, d, p, q)$. The instance $\succsim = >$ proves the same shift for $\mathsf{RRes}_{>}$ when $v \leq d$. The residual ensures that every event in the already-entered lower-bound region has q or an earlier p. If p occurs while the clock is still below threshold, the right side of weak Until terminates with that p; otherwise the clock reaches the threshold and the residual supplies the ordinary Release continuation. For $\geq d$, an event exactly at d is already in the constrained region; for $> d$, the prefix includes equality and only a strictly later event starts the region. The corresponding proofs are complete_RhatGe, rge_residual_shift, complete_RhatGt, and rgt_residual_shift.

D. OCCURRENCE-INDEXED STRUCTURAL INDUCTION

The completeness proof must recurse through a syntax tree while preserving the root formula's clock identities. Write $\mathsf{node}(root, \pi) = f$ when the path lookup at π in *root* returns f (the Rocq notation is `node_at root pi = Some f`). Let $\rho^\star = \mathsf{canonical_ext}(root, w)$. The strengthened theorem is:

Theorem V.1 (Canonical correctness at every syntactic occurrence)**.** *For every well-formed root formula, every path π with $\mathsf{node}(root, \pi) = f$, and every position i,*

$$w, i \models_{\mathrm{MTL}} f \quad \Longleftrightarrow \quad \rho^\star, i \models_{\mathrm{LTL}} \mathcal{T}_\pi(f).$$

The same extension $\rho^\star$ is constructed once from root and w and is used for all paths.

Proof. Proceed by structural induction on f. Boolean and untimed temporal cases follow from their semantics and the induction hypotheses. For a binary constructor, node_at_left_binary and node_at_right_binary locate the child formulas at the extended paths. Ordinary timed operators unfold to the derived Boolean/hatted definitions, leaving the eight primitive cases proved above. In each hatted case, the forward direction uses its canonical-completeness lemma and the reverse direction its policy-independent soundness lemma. The path locates the formula for induction; it is not part of the clock key. The Rocq theorem is canonical_encoding_all. □

Clock identity is structural equality of the primitive formula value, including constructor, operands, comparison, and real bound. Hence if $\mathsf{node}(root, \pi_1) = F = \mathsf{node}(root, \pi_2)$ for a primitive timed formula, both paths refer to the same clock and the same canonical trace. Rocq proves this as identical_nodes_share_clock and identical_nodes_share_canonical_trace. The theorem therefore uses one extension for the entire root formula, not a separate witness per occurrence or activation.

E. CLOCKED-LTL CORRECTNESS AND TBA COMPOSITION

Theorem V.2 (Correctness of the clock encoding)**.** *For every well-formed formula f and timed word w,*

$$\begin{aligned} w, 0 \models_{\mathrm{MTL}} f \quad \Longleftrightarrow \quad & \exists \rho.\ \mathsf{same_base}(\rho, w) \\ & \wedge\ \mathsf{clock_consistent}(\rho) \\ & \wedge\ \rho, 0 \models_{\mathrm{LTL}} \mathcal{T}(f). \end{aligned}$$

This is EncodingCorrect_proved. Its left-to-right direction uses the one canonical extension of Theorem V.1 at the root path. Its right-to-left direction uses the soundness induction for an arbitrary extension ρ; completeness cannot be used to reason about an arbitrary reset trace. Thus the quantifiers differ: every consistent satisfying extension is sound, whereas completeness constructs one extension.

Corollary V.3 (One clock per distinct primitive timed subformula)**.** *All occurrences of the same primitive timed formula use the same clock, and the encoding uses exactly the clocks in $Clock(f)$. The clock count is independent of the number of syntax-tree occurrences and dynamic activations.*

Proof. The translation is indexed by elements of $Clock(f)$, while T_uses_every_clock proves that each such clock occurs in $\mathcal{T}(f)$. The canonical trace of Theorem V.1 serves every activation of that formula. □

For a propositional Büchi automaton A over the extended atoms, let $\mathsf{compile_with}(A)$ denote relaxation followed by reset completion. Its correctness premise, Assumption IV.1, is language equivalence over the action-consistent propositional words, including valuations of the clock-related atoms that are not clock-consistent. The single composed theorem (Figure 5), conditional on the backend contract, is:

Theorem V.4 (Conditional MTL-to-TBA correctness)**.** *For every well-formed formula f, timed word w, and propositional Büchi automaton A satisfying Assumption IV.1,*

$$w, 0 \models_{\mathrm{MTL}} f \quad \Longleftrightarrow \quad \mathsf{TBA_accepts}(\mathsf{compile_with}(A), w).$$

In Rocq this is MTL_to_TBA_correct_with; backend language equivalence is an explicit premise.

Proof. For the forward direction, the encoding theorem supplies a canonical clock-consistent extension satisfying $\mathcal{T}(f)$. The backend premise gives an accepting propositional run. Relaxation carries the run to the relaxed automaton by relax_complete; reset completion then supplies the completed clock-consistent extension and accepting TBA run by complete_complete.

For the converse direction, an accepting completed run first projects to a run of the relaxed backend by complete_sound. At each position, reconstruct a propositional valuation satisfying the selected original backend cube: deleted negative extended literals are made false, surviving literals are preserved, and deleted irrelevant atoms are chosen to satisfy that cube. The same state sequence remains accepting. The backend contract gives $\mathcal{T}(f)$ on the reconstructed word. The relaxed word agrees on action atoms and only adds truth to relevant extended atoms, so positivity of $\mathcal{T}(f)$ transfers satisfaction to the relaxed word, as formalized by relax_sound. Applying the soundness direction of Theorem V.2 gives $w, 0 \models_{\mathrm{MTL}} f$. □

The convenience theorem MTL_to_TBA_correct chooses its backend using the project axiom LTL_TO_BUCHI_CORRECT. It is not assumption-free. The theorem displayed above instead takes backend equivalence as an explicit argument. This distinction is visible in `proof/ASSUMPTIONS.txt`.

F. ROCQ RESULT MAP

Table 3 maps the article's results to their checked declarations and assumptions. The primitive rows collect the four soundness and four canonical-completeness families, each with its strict variant.

The source-module aliases in Table 3 are:

Core. `MTL_to_TBA_Shared_Clock_Derived_Strict_Direct_Core.v`

Proof. `EncodingCorrect_Shared_Clock_Derived_Strict_Direct_Proof.v`

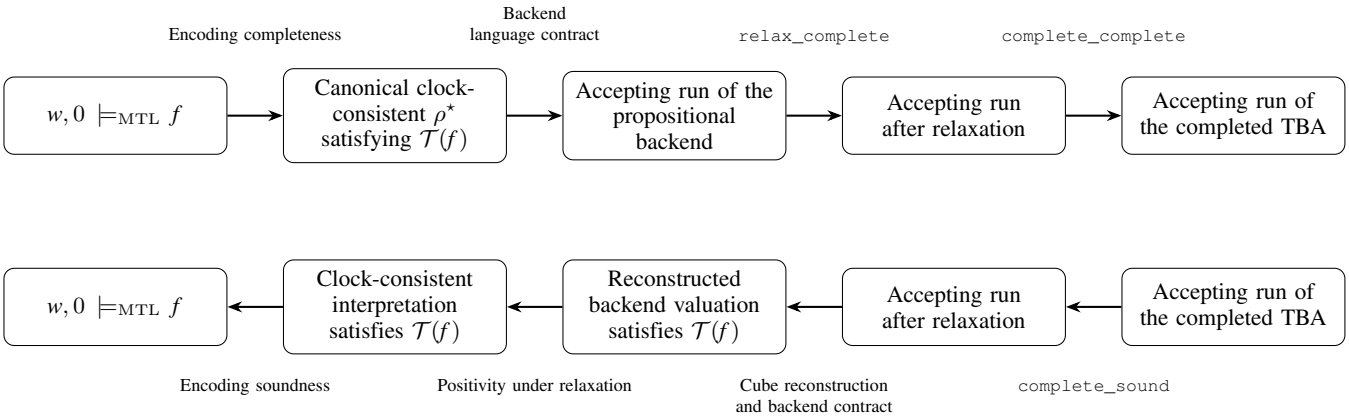


FIGURE 5. **Proof outline for Theorem V.4: completeness (top) and soundness (bottom), each using the backend equivalence contract.**

TABLE 3. **Map from the article's results to Rocq declarations, source modules (Core, Proof; file names below) and assumptions.**

Mathematical result	Rocq declaration(s)	Source module	Key dependency
Derived ordinary-operator semantics	Eight strict/non-strict compatibility lemmas	Core	Well-formed real bounds and strictly increasing timed words
Canonical correctness at each path; one extension	canonical_encoding_all; EncodingCorrect_proved	Proof	Standard Rocq real/classical and function-extensionality assumptions
Primitive soundness and canonical completeness	sound_UhatLe/Ge/Lt/Gt, sound_RhatLe/Ge/Lt/Gt; corresponding complete_* lemmas	Proof	Soundness: any consistent resets; completeness: canonical policy
Relaxation and reset completion	relax_sound, relax_complete, complete_sound, complete_complete	Core	Positive extended atoms; label side conditions
Conditional backend composition	MTL_to_TBA_correct_with	Proof	Explicit backend equivalence premise on action-consistent propositional words
Convenience backend theorem	MTL_to_TBA_correct	Proof	Project axiom plus standard Rocq assumptions

The assumptions report (the `Print Assumptions` output of Rocq for each main theorem, saved as `proof/ASSUMPTIONS.txt`) lists standard library dependencies for classical real comparison decisions, excluded middle, functional extensionality, and descriptions. These are distinct from the project-specific backend axiom. The theorem MTL_to_TBA_correct_with has no LTL_TO_BUCHI_CORRECT dependency; its backend premise is explicit. The convenience theorem does use that axiom.

The formal domain remains deliberately bounded: well-formed core formulas; real-valued mathematical parameters (not executable encodings of arbitrary reals); strictly increasing divergent timestamps; event-guarded automata without location invariants. Parsing, a concrete LTL backend implementation, Spot execution, graph import, and automaton printing remain outside the theorem. The sizes and checks of Section VI are measurements, not mechanized results.

The occurrence-indexed induction, residual invariants, and separate relaxation/reset-completion obligations expose the proof dependencies behind the language-preservation result. The development is supplied as inspectable Rocq source; the successful source replay procedure is documented with the artifact.

G. ALARM-RESPONSE PROOF INSTANCE

Consider $f_{alarm} = \mathsf{G}(alarm \Rightarrow \Diamond_{\le 3} handled)$, meaning every alarm must be handled within three time units. In proof/OverlappingResponse_Example.v, the formula is `MR MFalse (MOr (MNotAtom ALARM) (MUle 3 MTrue (MAtom HANDLED)))`, where `ALARM` and `HANDLED` are actions 1 and 2. Rocq checks that it is well formed and has one primitive timed subformula, $\widehat{\mathsf{U}}_{\le 3}\, \top\, handled$, whose obligations share clock x. The translation contains the negated event atom $\mathsf{LNAct}(alarm)$, which the tool prints as the explicit negation $\neg alarm$ of the event atom. Spot therefore receives

$$\mathsf{G}\big(\neg alarm \vee handled \vee \mathsf{X}((x \le 3 \wedge \mathsf{unch}(x)) \\ \mathsf{U}\,(x \le 3 \wedge handled))\big).$$

Spot 2.16's input and output are saved in examples/spot/alarm_response_backend.ltl and examples/spot/alarm_response_backend.dot [12]. The generated automaton, Fig. 6, has initial state 0 and accepting states $\{0, 1\}$. Rocq splits the disjunctive self-loop into two cubes. Lemma alarm_response_T_eq matches its input to the Rocq translation; alarm_response_spot_backend_contract proves that the transcribed automaton has exactly that LTL language on the action-consistent propositional words, which is the contract above. The resulting concrete TBA is alarm_response_spot_

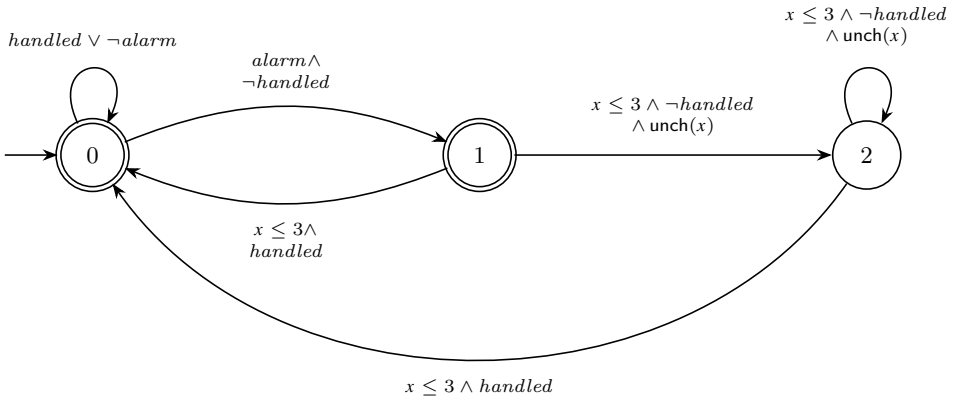


FIGURE 6. Propositional Büchi automaton generated by Spot for the alarm-response formula. Edges are labeled by the full Boolean condition on event and clock atoms. Double circles are accepting states, the incoming arrow marks the initial state, unch(x) is the preservation marker, and state numbers are Spot's.

tba. It has initial location 0, accepting locations $\{0, 1\}$, and clock x. Figure 7 shows the reset-completed automaton of the Rocq record. Each edge is labeled by its event, its clock guard, and its resets; guards are checked before resets, and an edge without a guard has none. The two edges from 0 to 0 of the record are drawn as one edge with two labels. The theorem alarm_response_spot_tba_correct instantiates Theorem V.4 and proves that this record accepts exactly the timed words satisfying f_{alarm}.

VI. EVALUATION

The theorem says nothing about the size of the automaton, or about how it compares with an independent implementation. This section measures the automaton $\mathsf{compile}(f) = \mathsf{compile_with}(A)$, where A is the automaton that Spot returns for $\mathcal{T}(f)$. It is obtained by the translation $\mathcal{T}$, Spot, relaxation, and reset completion, before any further processing, and no optimization is applied to it here. We ask three questions. Q1: how large is $\mathsf{compile}(f)$, compared with the automaton of CASAAL [11], which implements the construction of Li et al. [2]? Q2: on concrete formulas, do the languages agree with CASAAL's on the common restricted model (strictly increasing timestamps, mutually exclusive events), as far as a separately implemented timed-automaton checker, or evaluation on words, can tell? Q3: does the backend satisfy, on these formulas, the contract that the theorem assumes? The answers are measurements and tests.

a: Summary of outcomes.

Proved: the Rocq development of Section V; none of the numbers below is a proof. For Q1, the number of clocks of $\mathsf{compile}(f)$ coincides with CASAAL's on every exactly encoded configuration except F2; the nesting family N has CASAAL's number of locations, and locations and transitions are larger on most other configurations, for example 48 locations and 1625 transitions against 16 and 80 for R4 (R7: 576 locations and 296875 transitions against 128 and 1024), a growth that we attribute to Spot's automaton (Section VI-B). For Q2, the TChecker products P_1, P_2, S (Section VI-C) are empty on every configuration that completes (22 of the 31, two of them checked only for S) and on all 100 random formulas, which gives inclusion in CASAAL's language provided its two automata are complementary (not checkable in general: universality of a nondeterministic timed Büchi automaton is undecidable [13]); the word-level test finds no disagreement between $\mathsf{compile}(f)$ and our Python evaluation of the semantics on 26200 words, 25400 of them also compared with CASAAL. For Q3, no discrepancy with ltl2ba is found on 28 of the 31 configurations. Inconclusive: the larger configurations (R4–R8, N5–N8 for TChecker; R7, R8, N8 for the backend check) exceed the time limits and are reported as such.

A. SETUP

The tool `mtl2tba` parses a formula, computes $\mathcal{T}(f)$, calls Spot 2.16 (`ltl2tgba`), reads its LBTT output, and applies relaxation and reset completion. The functions $\mathcal{T}$, relaxation, and reset completion are OCaml code extracted from the Rocq definitions; the parser, the call to Spot, the LBTT reader, the printers, and extraction itself are not verified. The clocked-LTL formula is exactly $\mathcal{T}(f)$ as defined in Section III. The negated event atom $\mathsf{LNAct}(a)$ is printed for Spot as the negated literal `!"a"` of the atom of a, and `-compile-only` stops after reset completion, so that the reported time is that of $\mathsf{compile}(f)$. Measurements were taken on an Intel Core i7-8650U (1.9 GHz, 11.7 GB RAM) under Ubuntu 26.04 in WSL 2, with a limit of 600 s per tool run and per TChecker check (120 s for random formulas); the backend-contract runs of Spot and ltl2ba used 300 s, and their equivalence checks 600 s. The 31 configurations are fifteen specification patterns F1–F14 (F3b is F3 without hat), conjunctions Rk of k bounded responses $\square(a_i \Rightarrow \Diamond_{\leq 5} b_i)$ for $k \in 1..8$, and nestings Nk of k bounded eventualities $\Diamond_{\leq 5}(p_1 \wedge \Diamond_{\leq 5}(p_2 \wedge \cdots))$. Scripts, inputs, CASAAL outputs, and raw results are in the artifact (`evaluation/`).

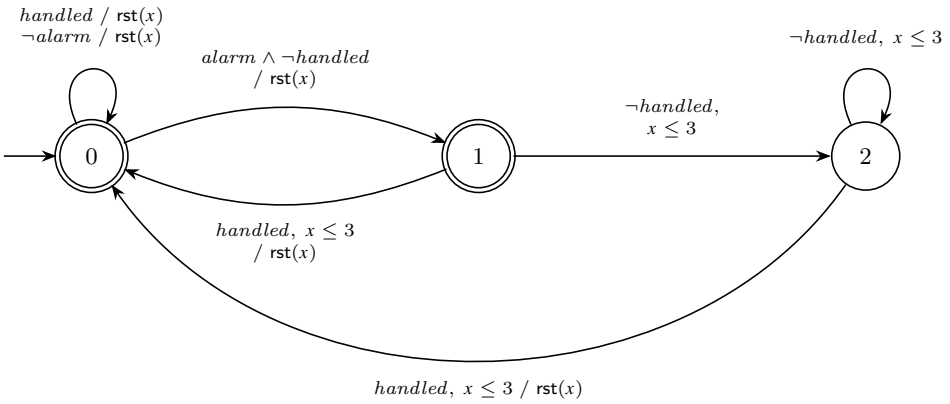


FIGURE 7. Timed automaton of the alarm-response Rocq record after relaxation and reset completion. Edges are labeled by event, clock guard and, after a slash, the clocks reset; double circles are accepting states. The Graphviz DOT graph-to-record correspondence remains outside the Rocq proof.

TABLE 4. The specification patterns F1–F14 (and F3b, F3 without the hat). $\widehat{\Box}, \widehat{\Diamond}$ are the hatted operators of Section II; R*k* and N*k* are defined in the text.

ID	Pattern	Formula
F1	bounded response	$\Box(p \Rightarrow \Diamond_{\leq 5} q)$
F2	bounded invariance	$\Box(p \Rightarrow \Box_{\leq 3} q)$
F3	sporadicity (hatted)	$\Box(e \Rightarrow \widehat{\Box}_{<2} \neg e)$
F3b	sporadicity, no hat	$\Box(e \Rightarrow \Box_{<2} \neg e)$
F4	time-bounded Until	$\Box(p \Rightarrow (p \,\mathsf{U}_{\leq 4}\, q))$
F5	minimum-delay response	$\Box(p \Rightarrow \Diamond_{\geq 2} q)$
F6	strict minimum-delay response	$\Box(p \Rightarrow \Diamond_{>2} q)$
F7	bounded recurrence	$\Box\Diamond_{\leq 5} p$
F8	recurrence with minimum delay	$\Box\Diamond_{\geq 2} p$
F9	nested Until	$\Diamond_{\leq 5}(p \,\mathsf{U}_{\leq 2}\, q)$
F10	nested Until with invariance	$\Diamond_{\leq 5}((\Box_{<3}(p \vee q)) \,\mathsf{U}_{\leq 2}\, q)$
F11	running example	$\Diamond_{\leq 5} p \wedge \Box_{<2} \neg p$
F12	response within a window	$\Box(p \Rightarrow (\Diamond_{\leq 5} q \wedge \Box_{<2} \neg q))$
F13	periodicity (hatted)	$e \wedge \Box(e \Rightarrow (\widehat{\Box}_{<2} \neg e \wedge \widehat{\Diamond}_{\leq 2} e))$
F14	closeness	$\Box(e_1 \Rightarrow (\Diamond_{<2}(e_2 \wedge \Diamond_{<3} e_3) \vee \Box(\neg e_2 \wedge \neg e_3)))$

B. SIZE OF COMPILE(F) AND CASAAL REFERENCE

Table 4 lists the patterns F1–F14, and Table 5 gives, for each configuration, the reachable locations, transitions, and clocks of compile(f), and the size of CASAAL's output. CASAAL was run with mutually exclusive events, which matches our one-event-per-position words. The two counts of transitions are not directly comparable: CASAAL labels a transition with a Boolean formula over propositions, whereas Spot's output, and hence compile(f), has one transition per cube over the propositions and clock literals. In R4, for example, compile(f) has 48 locations and 1625 transitions, and CASAAL 16 locations and 80 symbolic transitions; both use four clocks. Column Grp. T counts the transitions of compile(f) after grouping edges with the same source, target, guard, and resets into one edge labeled by the union of their events; it is a way of counting and changes no automaton. Grouping reduces R4 from 1625 to 768 and N8 from 166 to 54 transitions, but a gap remains (CASAAL: 80 and 19), so the difference is not explained by the representation of labels alone.

TABLE 5. Automaton compile(*f*) before any optimization, for the 31 configurations of Section VI (patterns in Table 4; R*k*, N*k* in the text). L/T/X: reachable locations/transitions/clocks. Grp. T: transitions after grouping edges with the same source, target, guard, and resets (a count, not a transformation). CASAAL: locations/symbolic transitions/clocks, mutually exclusive events (n/a: no exact encoding). P_1, P_2, S: TChecker products ("empty": all three empty; "S only": only S computed, hatted formulas; "timeout": undecided within 600 s). Backend: Spot's automaton against ltl2ba with Spot's tools ("yes": both products empty; the other entries are timeouts, not discrepancies: "check timeout", equivalence check above 600 s; "ltl2ba timeout" and "tool timeout" (Spot), both above 300 s). In L/T/X, "timeout": Spot did not finish, so `compile` did not run. "–": not run. Tools: Spot 2.16, ltl2ba 1.3, TChecker 0.8.

ID	L/T/X	Grp. T	CASAAL	P_1, P_2, S	Backend
F1	3/7/1	6	2/4/1	empty	yes
F2	2/5/1	5	1/1/0	empty	yes
F3	2/5/1	5	n/a	S only	yes
F3b	1/1/0	1	1/1/0	empty	yes
F4	3/7/1	6	2/4/1	empty	yes
F5	7/38/1	34	4/15/1	empty	yes
F6	7/38/1	34	4/15/1	empty	yes
F7	3/6/1	6	2/4/1	empty	yes
F8	5/15/1	15	2/4/1	empty	yes
F9	4/9/2	9	4/9/2	empty	yes
F10	5/13/3	12	5/12/3	empty	yes
F11	4/7/2	7	4/7/2	empty	yes
F12	4/13/2	13	3/9/2	empty	yes
F13	4/9/2	9	n/a	S only	yes
F14	7/31/2	23	8/18/2	empty	yes
R1	3/7/1	6	2/4/1	empty	yes
R2	8/45/2	32	4/12/2	empty	yes
R3	20/275/3	160	8/32/3	empty	yes
R4	48/1625/4	768	16/80/4	timeout	yes
R5	112/9375/5	3584	32/192/5	timeout	yes
R6	256/53125/6	16384	64/448/6	timeout	yes
R7	576/296875/7	73728	128/1024/7	timeout	check timeout
R8	timeout	–	256/2304/8	–	tool timeout
N1	3/5/1	5	3/5/1	empty	yes
N2	4/11/2	9	4/7/2	empty	yes
N3	5/21/3	14	5/9/3	empty	yes
N4	6/36/4	20	6/11/4	empty	yes
N5	7/57/5	27	7/13/5	timeout	yes
N6	8/85/6	35	8/15/6	timeout	yes
N7	9/121/7	44	9/17/7	timeout	yes
N8	10/166/8	54	10/19/8	P1 timeout; P2,S –	ltl2ba timeout

The number of clocks coincides with CASAAL's on every configuration that CASAAL encodes exactly, except F2: under mutual exclusion, $\Box(p \Rightarrow \Box_{\leq 3} q)$ forbids p altogether,

and CASAAL produces a one-state automaton without a clock, whereas compile(f) keeps two locations and one clock. This agrees with the clock discipline of the construction: one clock per structurally distinct primitive timed subformula. Locations and transitions are larger than CASAAL's on most configurations, sometimes by a large factor (R4, R5); the nesting family N has the same number of locations as CASAAL. The growth in the R family appears to come from Spot's automaton for the conjunction of k clocked-LTL clauses, before relaxation (Spot's automata for R4–R6 have 768, 3584, and 16384 transitions). On R4 to R7, Spot's translation dominates the time up to compile(f): Spot takes 0.12, 2.1, 5.4, and 90 s of a total (parsing, Spot, relaxation, reset completion) of 0.18, 2.2, 6.1, and 96 s, and the construction proper (relaxation and reset completion) takes at most 6 s (R7; `evaluation/stage_times.tsv`). For R8, Spot does not finish within 300 s. A complete run of `mtl2tba` up to compile(f), which also writes the DOT file, takes less than one second for every configuration except R5 (3.0 s), R6 (9.2 s), R7 (113 s), N6 (1.4 s), N7 (16 s), and N8 (85 s). These numbers describe the unoptimized output of the extracted construction; they are not a comparison of tools. Reducing these sizes by post-processing is left to follow-up work.

Printing the negated event atom explicitly does not change R and N, whose formulas contain no negated event, and reduces F11–F14 (for example F11 from 5/11/2 to 4/7/2). On the 100 random formulas it changes 26 automata, all smaller: the total of locations falls from 793 to 660, that of transitions from 3681 to 2305, and that of clocks from 163 to 159. The independent printing leaves cubes that require both a and $\neg a$, which no one-event-per-position word satisfies; the explicit printing does not produce them.

C. LANGUAGE CHECKS WITH TCHECKER

For a formula φ, let O and O' be compile(φ) and compile($\neg\varphi$), and C, C' CASAAL's automata for φ and $\neg\varphi$. TChecker 0.8 [14] decides the emptiness of the products $P_1 = O \times C'$, $P_2 = C \times O'$, and $S = O \times O'$, built over infinite words with one event per position, strictly increasing and divergent timestamps, and clocks starting at 0. P_1 and P_2 empty mean that O is disjoint from C' and C from O'; if CASAAL's two automata are complementary, which is not checked, this gives $L(O) \subseteq L(C)$ and $L(O') \subseteq L(C')$. The converse inclusions would additionally require that O and O' together cover all words, which is not checked either. The checks therefore establish inclusions, not equality. S empty means that our automata for φ and $\neg\varphi$ never accept the same word. A fourth product $O \times C$ is a sanity check: it must be non-empty when φ is satisfiable.

On the benchmark configurations, P_1, P_2, and S are empty for every configuration that completes within the limits (F1–F14 and F3b, except the hatted F3 and F13, which CASAAL cannot express; R1–R3; and N1–N4); for F3 and F13 only S is checked, and it is empty. For R4–R7 and N5–N7 the three checks exceed 600 s. At 1800 s, run singly, P_2 of R4 is empty, while P_1, S of R4 and all three checks of N5 still time out (which may also mean memory exhaustion). For R8 no automaton is built within the limit. For N8, O is built (85 s), but P_1 exceeds 600 s and O' is not built within the limit, so P_2 and S are not run (inconclusive). The sanity product is non-empty in every completed benchmark configuration.

The artifact also contains 100 random formulas of depth at most three over three propositions, generated with their CASAAL automata. For all 100 of them, P_1, P_2, and S are empty, with no timeout; for five of these, the sanity product is empty as well, and inspection shows that each of these five formulas is unsatisfiable under mutual exclusion (for example $q \wedge \Diamond_{>2} p \wedge r$), so the other 95 passes are non-vacuous.

One formula, X74 $= \Diamond(q \wedge r) \Rightarrow \neg p$, first gave a non-empty P_1. Its negation $\Diamond(q \wedge r) \wedge p$ is unsatisfiable under mutual exclusion, and our automaton for it has no accepting location. CASAAL prints its automaton for the negation without an acceptance mark, and the conversion script read an automaton without marks as accepting every run. The state labels of that automaton show an obligation $\mathsf{F}(q \wedge r)$ that no transition discharges. The converter now reads an automaton without marks as follows: a state whose label contains an unbounded eventuality (F or U without an upper bound) is not accepting, and the language is empty if no state is accepting; an automaton without such labels keeps the previous reading. This changes the verdict of X74 only; the other 15 of the 16 affected CASAAL automata, and all other formulas, give the same results as before. The reading is our interpretation of CASAAL's output, adopted after the discrepancy on X74 (the other 15 cases give the same results under either reading), not a documented convention of the tool; a regression test is in the artifact (`evaluation/test_tck_product_acceptance.py`).

D. AGREEMENT ON ULTIMATELY PERIODIC WORDS

The TChecker checks establish inclusions and time out on the larger configurations, so we add a test of equality on finitely many words. For each configuration we draw random ultimately periodic timed words uv^ω (a prefix of 0–4 and a loop of 1–5 events, integer timestamps in half units, strictly increasing and divergent; half of the words follow the order of the propositions of the formula), and compare three verdicts: the formula under the pointwise semantics above, evaluated by a Python implementation that we wrote independently of the translator (it is not extracted from, nor checked against, the Rocq definitions); acceptance by compile(f); and acceptance by CASAAL's automaton. Clocks of CASAAL start at 0; for compile(f) initial clock values are tried on a grid when it has at most three clocks, since the theorem quantifies them existentially; with more clocks, all clocks start at 0. On 28 benchmark configurations (all but R6–R8; 400 words each, 11200 in all, 4495 satisfying the formula) and on the 100 random formulas (150 words each, 15000 in all, 8273 satisfying it), the semantics and compile(f) agree on every word. CASAAL is included on 26 of the 28 benchmark configurations (not on the hatted F3 and F13) and on all random formulas, and agrees on every word there as well: 10400 plus 15000 words, 25400

three-way comparisons; the other 800 words compare only the semantics and the automaton. As a negative control, the same checker finds disagreements for F1 against the formulas with bound 4 instead of 5 (11 of 300 words) and with a strict bound (5 of 300). The test has little weight for some formulas: no word of N8 satisfies the formula among the random ones, so for N5–N8 we added five hand-built words each (accepted and rejected; all agree), and nine random formulas have fewer than five satisfying and fourteen fewer than five violating words. The words are finitely many and short; this is a test, not a proof of equality.

E. BACKEND CONTRACT

Theorem MTL_to_TBA_correct_with assumes that the propositional automaton accepts exactly the models of $\mathcal{T}(f)$. For each configuration we saved the formula given to Spot and the LBTT automaton A it returned, and checked the contract with Spot's `autfilt`, using an independent translator, ltl2ba 1.3: $A \times \mathrm{ltl2ba}(\neg\mathcal{T}(f))$ and $\overline{A} \times \mathrm{ltl2ba}(\mathcal{T}(f))$, where $\overline{A}$ is the complement of A, must both be empty. No discrepancy is found on 28 of the 31 configurations; for R7 the equivalence check exceeds 600 s, for N8 ltl2ba exceeds 300 s, and for R8 Spot does not finish within 300 s. The acceptance marks of Spot's automaton are converted to the Büchi acceptance of the Rocq record by the unverified LBTT reader, whose labels and acceptance sets were checked only through the product tests above. On every configuration where Spot produced an automaton, the number of states and transitions read by the LBTT reader of `mtl2tba` equals the number reported by `autfilt` on the same file. These checks test the contract on these inputs; they do not prove it, and they use Spot's own tools for part of the check.

F. HATTED OPERATORS

F3 and F13 use the hatted operators of Section II. CASAAL has no hatted operator; its closest encoding of F3, $\Box(e \Rightarrow \mathsf{X}\Box_{\leq 2}\neg e)$, measures the window from the next event rather than from the triggering one (Fig. 2) and is not equivalent, so we do not compare the two languages. The two readings give different automata. For F3 and F13 the evidence is limited to S and to agreement with our Python evaluator, since no independent tool expresses them. Under mutual exclusion the non-strict window of the unhatted F3b contains the triggering event, so F3b forbids e altogether: $\mathsf{compile}(f)$ has one location, one transition, and no clock. The window of F3 starts after the triggering event but is measured from its timestamp, and $\mathsf{compile}(f)$ has two locations, five transitions, and one clock x, reset on each e; after an e, a further e is accepted only with the guard $x > 2$.

G. THREATS TO VALIDITY

The configurations and random formulas are small, of our choosing, and use integer bounds only, so real-valued bounds are covered by the mathematical theorem but not by the extracted tool or by any test. The TChecker products and, for formulas with more than three clocks, the word-level test start all clocks at 0, whereas the theorem quantifies the initial clock values existentially. The R family shows that the size of Spot's automaton grows quickly with the number of conjuncts. The CASAAL automata were produced earlier and not regenerated; CASAAL's integer bounds, nondecreasing time, and acceptance printing differ from our model, and the inclusions depend on the unchecked complementarity of its automata. The TChecker products, the word-level test, and the conversion scripts are unverified code. Timeouts are reported as inconclusive.

VII. ARTIFACT AND REPRODUCIBILITY

The artifact is available at https://github.com/efares1/rocq-mtl-to-tba-translation; the revision described here is the release tagged `v1.2`. It contains the Rocq sources, the assumptions report, the tool `mtl2tba`, the evaluation scripts with their inputs and raw results, the Spot input and graph of the alarm-response example, and the article source. The proofs build with Rocq 9.0.1: `make` in `proof/` compiles the core, the correctness development, and the alarm example; `proof/CheckAssumptions.v` prints the assumptions of the main theorems, which are recorded in `proof/ASSUMPTIONS.txt`; and `coqchk` re-checks the compiled objects. On Windows, `reproduce-ieee-access.ps1` performs these steps, runs the two Python assurance scripts, and builds the PDF. The scripts are executable audits, not verified components: `examples/assurance/check_spot_transcription.py` compares the Spot graph with the Rocq record of the alarm-response instance, up to state renaming, including edge guards (with mutation self-tests), and `examples/assurance/reset_policy_scenarios.py` checks the finite reset-policy scenarios of Table 1.

The main checked claims have direct source declarations: EncodingCorrect_proved and canonical_encoding_all establish the formula-to-clocked-LTL equivalence; MTL_to_TBA_correct_with composes this result with the explicit backend contract on action-consistent propositional words; alarm_response_spot_tba_correct is the concrete alarm instance. The primitive theorem families, relaxation lemmas, reset-completion lemmas, and their assumptions are mapped in Table 3.

The evaluation of Section VI is reproduced, under Linux or WSL with OCaml, Spot 2.16, TChecker 0.8, and ltl2ba 1.3, by building the tool in `tool/` and running `evaluation/run_compiled_eval.sh` and `evaluation/run_spot_validation.sh`; `evaluation/make_compiled_table.py` regenerates Table 5 from the raw results. The word-level test is `evaluation/lasso_check.py`, run on the automata written by `evaluation/dump_compile_dot.sh`, and `evaluation/grouped_edges.py` computes the grouped transition counts.

VIII. RELATED WORK

Pointwise MTL has extensive decidability and expressiveness results [8]–[10], [15], [16]. Unrestricted dense-time MTL

TABLE 6. Representative mathematical results and mechanization evidence for timed-logic-to-automata constructions. The rows have different logics, semantics, targets, and proof scopes; they are not a performance comparison.

Work	Semantic scope	Existing correctness and construction	Mechanization and implementation evidence
Bulychev [1]	Pointwise non-Zeno infinite words; integer bounds, all four comparisons, nondecreasing timestamps.	Theorems 1–2 prove exact automaton languages for safety/co-safety. The construction extends the classical tableau LTL-to-Büchi translation directly, with timed-subformula clocks, obligations, and reset/unchanged markers.	Mathematical proofs and controller-synthesis tooling for the stated fragments.
Li et al. [2]	Full $\mathrm{MTL}_{0,\infty}$; pointwise non-Zeno words, integer bounds, all four comparisons, nondecreasing timestamps, zero initial clocks.	Establishes exact language equality for the timed-Büchi construction (closure, extended formulas, degeneralization) and language preservation under best-choice resets.	CASAAL implements the workflow and deterministic approximations [11]; the proof is mathematical.
Xu and Miao [3]	MITL formulas mapped to fair timed automata; exact semantic relation to this paper's pointwise event-word model is not compared.	One tester automaton per subformula, with local clocks measuring elapsed time against the metric interval, composed in parallel; no intermediate untimed LTL stage.	The construction and its implementation in PVS.
This paper	Well-formed Rocq formulas with real bounds, strictly increasing timestamps, existential nonnegative initial clocks.	Translation factored through clocked LTL and a standard LTL-to-Büchi backend. EncodingCorrect_ proved proves clocked-LTL equivalence for all eight primitive cases; TBA composition is conditional on backend equivalence.	Rocq-checked proof and one proved contract for a manual Spot transcription; tool evaluation with extracted code (Section VI).

has no general effective translation to finite ordinary timed automata; bounded-variability and alternating-automaton results have different scopes [17], [18]. $\mathrm{MTL}_{0,\infty}$ retains common response and separation properties [1], [16]. Our hatted operators make the timing reference explicit in the logic and proof.

Translations of timed logics. Li et al. establish the full-fragment mathematical language-equivalence result. Our overlapping-activation proof checks completeness for the present formal model; we do not infer that prior constructions lacked such handling.

Bulychev et al. state that their translation uses "the classical tableau-based translation from linear-time temporal formulas into Büchi automata" with two modifications: no Büchi acceptance condition for the safety and co-safety fragments, and an auxiliary clock for each timed subformula. Li et al. proceed by a closure, extended formulas that carry the clock bookkeeping, a normal form, a transition-based timed Büchi automaton, and a degeneralization step. In both, as we read them, the timed expansion and the Büchi construction are one hand-built construction. Ours differs in architecture: the formula is encoded into untimed clocked LTL, any LTL-to-Büchi backend may then be used, and its correctness enters the composed theorem as an explicit contract. This factoring is what the Rocq development exploits, and it is also why the backend premise is separate from the timed argument. To the best of our knowledge, none of the translations from MTL or MITL to timed automata that we examined (the two above, the testers of Maler et al., MightyL, MightyPPL, and Xu and Miao) is presented in this form; we did not examine every cited work in the same depth.

Xu and Miao describe a PVS-based transformation from MITL formulas to fair timed automata, including construction principles and implementation details [3]. Their construction builds one tester automaton per subformula, each with local clocks that measure the elapsed time against the metric interval, and composes the testers in parallel; it has no intermediate untimed LTL stage. This is an earlier proof-assistant-based timed-logic translation; our scope-specific comparison concerns the pointwise $\mathrm{MTL}_{0,\infty}$ construction formalized here.

MITL translations use temporal testers, automata networks, or one-clock alternating automata [18]–[20]; MightyL and MITL2Timed provide UPPAAL-oriented tools for that distinct logic [21], [22]. MightyL, for example, outputs a network of timed automata, one per temporal subformula, generalizing the Gastin–Oddoux LTL-to-Büchi construction; as described in [21], it does not call an LTL-to-Büchi tool on a timed formula. Obligation-merging techniques are related, while our one-sided residual predicates support the checked completeness proof.

Clocked temporal logic and subformula clocks. The clocked linear temporal logic extension CLTLoc adds clock comparisons and reset predicates and has been used to encode metric logics [23]. Our similar propositional vocabulary is an automaton-synthesis intermediate: the backend treats clock tests and reset markers as propositions. Positivity gives upward closure, and reset completion restores timed meaning; hence the explicit relaxation proof.

Event-clock logics associate clocks with subformulas to measure time since or until events [17], [24], [25]. Formula-indexed clocks are established; our proof checks that one clock per distinct timed formula preserves language across repeated activations and all cases (Section V-D).

Recent automata-based real-time verification. Recent neighboring work addresses different settings: an improved one-clock alternating MTL automaton for finite timed words with clock deactivation and zone analysis [26]; and TEMPORA's pointwise MITL model checking with past and future

modalities [27]. These do not translate our $\mathrm{MTL}_{0,\infty}$ fragment to infinite-word TBAs. Ho et al.'s MightyPPL translates pointwise MITL with past and Pnueli modalities over finite and infinite timed words; it gives mathematical construction results and proof sketches alongside a complete implementation whose testers sequentialize overlapping obligations [28]. A companion preprint [29] describes an implementation for existentially punctual metric temporal logic with past and Pnueli modalities (MTLPPL). Its term *one-sided MTL* denotes the subfragment without past or Pnueli, whereas here it describes interval shapes $[0, d]$ and $[d, \infty)$. Any syntactic containment comparison is limited to conventional source formulas after zero-bound reduction and with integer bounds; it excludes hatted abstract syntax tree (AST) constructors. The QEST fragment also permits punctual and non-one-sided intervals in its stated positions. Its punctual-response example does not imply support for unrestricted punctual MTL; differing bounds and timed-word conventions preclude a shared-input semantic comparison.

These tools (TEMPORA, MightyPPL, and the alternating-automaton work above) address different fragments or output models; this paper neither re-verifies them nor makes a shared-input, coverage, clock-count, or performance comparison with them. The Rocq development is not integrated with CASAAL, MightyPPL, or another checker.

Mechanized translations and trusted backends. Proof-assistant work already verifies LTL-to-Büchi constructions and model checkers [5], [30]–[32]. Wang et al. verify an MLTL-to-regular-expression transformation in Isabelle/HOL, export executable code, and compare it with existing implementations [6]. Chattopadhyay and Mamouras formalize in Coq a compositional construction of online monitors for past-time MTL over a discrete temporal domain, using quantitative semantics and Moore machines; they also extract OCaml monitors. This is a distinct monitoring target and semantic setting from the infinite-word TBA construction studied here [7]. For metric logics, Roohi and Viswanathan use PVS to correct an unsound release semantics in prior MITL decision procedures and prove supporting results for an adapted timed-automata construction under continuous-time signal semantics [4]. Xu and Miao's PVS-based construction is described above. Together, these works establish substantial proof-assistant precedents for related metric logics, monitors, and automata; this paper adds a Rocq-checked correctness development for the specified pointwise infinite-word TBA construction.

Event-B refinement patterns. The context of this work is the generation of Event-B patterns from timed formulas: the TBA translation studied here is intended as the input of a later pattern-generation step. Our earlier work on Event-B patterns covers their DSL-based generation [33], their derivation through weakest preconditions [34], and predicate transformers [35]. The present paper covers the MTL-to-TBA stage only and gives no TBA-to-Event-B transformation.

IX. CONCLUSION

This paper provides a Rocq-checked soundness-and-completeness development for the specified pointwise $\mathrm{MTL}_{0,\infty}$-to-TBA construction. Its proof covers all eight primitive cases, soundness for arbitrary clock-consistent reset traces, and completeness using one canonical trace across repeated activations of formula-keyed clocks. The occurrence-indexed induction connects each syntactic occurrence to its translation while preserving the identity of a shared clock. This supplies a checked account of the obligations that a mathematical argument must discharge when several timed requirements overlap.

Rocq also checks relaxation and reset completion and composes them with an explicit propositional backend language-equivalence contract. The resulting theorem makes its backend dependency inspectable instead of hiding it in an assurance claim. This work builds on the mathematical language-preservation results of Li et al. and adjacent proof-assistant work, while contributing a reformulation that factors the translation through untimed clocked LTL and a standard LTL-to-Büchi backend, and a machine-checked derivation for this construction and the semantic model defined here [2]–[4].

The evaluation of the extracted construction found no disagreement with CASAAL on the checked configurations and words, and no discrepancy of the backend contract on 28 of the 31 configurations; the larger configurations are inconclusive.

The theorem applies to the stated pointwise semantics and composes through an explicit backend contract. The contract is discharged only for a manually transcribed backend record in the alarm-response instance, and Spot's graph generation/import and zero-bound preprocessing remain outside the checked boundary. Future work can extend the mechanization to these components and connect the semantic theorem to a verified executable backend.

The proof artifact and assumptions report are described in Section VII.

REFERENCES

[1] P. Bulychev, A. David, K. Larsen, and G. Li, "Efficient controller synthesis for a fragment of MTL(0,∞)," *Acta Informatica*, vol. 51, no. 3-4, pp. 165–192, Jan. 2014.

[2] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, "Practical controller synthesis for $\mathrm{MTL}_{0,\infty}$," in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111.

[3] Q.-G. Xu and H.-K. Miao, "Timed automata for metric interval temporal logic formulae in prototype verification system," *Journal of Shanghai University (English Edition)*, vol. 12, no. 4, pp. 339–346, 2008.

[4] N. Roohi and M. Viswanathan, "Revisiting MITL to fix decision procedures," in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2018)*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 474–494.

[5] A. Schimpf, S. Merz, and J.-G. Smaus, "Construction of Büchi automata for LTL model checking verified in Isabelle/HOL," in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.

[6] Z. Wang, K. Kosaian, and K. Y. Rozier, "Formally verifying a transformation from MLTL formulas to regular expressions," in *Tools and Algorithms*

for the Construction and Analysis of Systems (TACAS 2025), ser. Lecture Notes in Computer Science, vol. 15696. Springer, 2025, pp. 254–275.
[7] A. Chattopadhyay and K. Mamouras, "A verified online monitor for metric temporal logic with quantitative semantics," in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 383–403.
[8] D. D'Souza and P. Prabhakar, "On the expressiveness of MTL in the pointwise and continuous semantics," *International Journal on Software Tools for Technology Transfer*, vol. 9, no. 1, pp. 1–4, 2007.
[9] J. Ouaknine and J. Worrell, "On the decidability of metric temporal logic," in *20th Annual IEEE Symposium on Logic in Computer Science (LICS 2005)*. IEEE Computer Society, 2005, pp. 188–197.
[10] P. Bouyer, F. Chevalier, and N. Markey, "On the expressiveness of TPTL and MTL," in *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2005)*, ser. Lecture Notes in Computer Science, vol. 3821. Springer, 2005, pp. 432–443.
[11] G. Li, "CASAAL: A tool for translating $\mathrm{MTL}_{0,\infty}$ into timed Büchi automata," Online, 2017, available at the CASAAL tool page. [Online]. Available: https://lcs.ios.ac.cn/~ligy/tools/CASAAL/
[12] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, E. Renault, and L. Xu, "Spot 2.0 — a framework for LTL and ω-automata manipulation," in *Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA'16)*, ser. Lecture Notes in Computer Science, vol. 9938. Springer, Oct. 2016, pp. 122–129.
[13] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.
[14] F. Herbreteau and G. Point, "TChecker: model checker for timed automata," https://github.com/ticktac-project/tchecker, 2025, version 0.8.
[15] R. Koymans, "Specifying real-time properties with metric temporal logic," *Real Time Syst.*, vol. 2, no. 4, pp. 255–299, 1990.
[16] R. Alur, T. Feder, and T. A. Henzinger, "The benefits of relaxing punctuality," *Journal of the ACM (JACM)*, vol. 43, no. 1, pp. 116–146, 1996.
[17] D. Ničković and N. Piterman, "From MTL to deterministic timed automata," in *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)*, ser. Lecture Notes in Computer Science, vol. 6246. Springer, 2010, pp. 152–167.
[18] T. Brihaye, M. Estiévenart, and G. Geeraerts, "On MITL and alternating timed automata over infinite words," 2014. [Online]. Available: https://arxiv.org/abs/1406.4395
[19] O. Maler, D. Nickovic, and A. Pnueli, "From MITL to timed automata," in *Formal Modeling and Analysis of Timed Systems (FORMATS 2006)*, ser. Lecture Notes in Computer Science, vol. 4202. Springer, 2006, pp. 274–289.
[20] T. Ferrère, O. Maler, D. Ničković, and A. Pnueli, "From real-time logic to timed automata," *Journal of the ACM*, vol. 66, no. 3, 2019, art. no. 19.
[21] T. Brihaye, G. Geeraerts, H.-M. Ho, and B. Monmege, "MightyL: A compositional translation from MITL to timed automata," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds. Cham: Springer International Publishing, 2017, pp. 421–440.
[22] "MITL2Timed: Modifying Oddoux and Gastin LTL2BA to timed automata generation," GitHub repository. [Online]. Available: https://github.com/yzh89/MITL2Timed
[23] M. M. Bersani, M. Rossi, and P. San Pietro, "A tool for deciding the satisfiability of continuous-time metric temporal logic," *Acta Informatica*, vol. 53, no. 2, pp. 171–206, 2016.
[24] R. Alur, L. Fix, and T. A. Henzinger, "Event-clock automata: A determinizable class of timed automata," *Theoretical Computer Science*, vol. 211, no. 1–2, pp. 253–273, 1999.
[25] J.-F. Raskin and P.-Y. Schobbens, "The logic of event clocks: Decidability, complexity and expressiveness," *Journal of Automata, Languages and Combinatorics*, vol. 4, no. 3, pp. 247–286, 1999.
[26] P. Bouyer, B. Srivathsan, and V. Vishwanath, "Model-checking real-time systems: Revisiting the alternating automaton route," in *Foundations of Software Science and Computation Structures (FoSSaCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15691. Springer Nature Switzerland, 2025, pp. 399–421.
[27] S. Akshay, P. Contractor, P. Gastin, R. Govind, and B. Srivathsan, "TEMPORA: Efficient verification of metric temporal properties with past in pointwise semantics," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 640–659.
[28] H.-M. Ho, S. N. Krishna, K. Madnani, R. Majumdar, and P. Pandya, "MightyPPL: Model checking MITL with past and Pnueli modalities," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 457–479.
[29] ——, "MightyPPL: Towards model checking MTL," 2026. [Online]. Available: https://arxiv.org/abs/2609.19073
[30] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, "A fully verified executable LTL model checker," in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
[31] A. Schimpf and P. Lammich, "Converting linear-time temporal logic to generalized Büchi automata," *Archive of Formal Proofs*, 2014, https://isa-afp.org/entries/LTL_to_GBA.html, formal proof development.
[32] B. Seidl and S. Sickert, "A compositional and unified translation of LTL into ω-automata," Archive of Formal Proofs, https://isa-afp.org/entries/LTL_Master_Theorem.html, 2019.
[33] E. Fares, J.-P. Bodeveix, and M. Filali, "Pattern-based refinement generation through domain specific languages," in *Rigorous State-Based Methods (ABZ 2023)*, ser. Lecture Notes in Computer Science, vol. 14010. Springer, 2023, pp. 35–42.
[34] ——, "Correct pattern-based development through refinements and weakest preconditions calculus," in *Formal Aspects of Component Software (FACS 2024)*, ser. Lecture Notes in Computer Science, vol. 15189. Springer, 2024, pp. 59–78.
[35] ——, "Correct pattern-based development through refinements and predicate transformers," *Science of Computer Programming*, vol. 255, p. 103550, 2027, forthcoming; scheduled for January 2027.

ELIE FARES Elie Fares is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include formal methods, refinement-based development and mechanized verification.

JEAN-PAUL BODEVEIX Jean-Paul Bodeveix is with IRIT, Université de Toulouse, France. His research interests include formal methods, semantic models, refinement-based correct-by-construction development, and proof assistants.

HUSSAIN AL-AQRABI Hussain Al-Aqrabi is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include cybersecurity and cloud security.

AZAR SALAMI Azar Salami is with the Higher Colleges of Technology, Sharjah, UAE. His research interests include mathematics and formal methods.